1. Title

Testing and Code Coverage for ResQNow Backend – Sprint 01

2. Objective

- To verify that all backend functionalities work as intended.
- To measure code coverage and identify untested areas for improvement.
- To ensure that APIs such as user registration, authentication, and incident reporting are reliable.

3. Tools Used

- **Jest**: Unit and integration testing framework for Node.js.
- **dotenv**: For loading environment variables in test environment.
- **MongoDB Atlas**: Backend database for storing users and incidents.
- **Postman**: Manual testing for API endpoints.

4. Methodology

1. Write unit tests for user, auth, and incident controllers.
2. Run tests using npm test.
3. Generate a coverage report using npm run coverage.
4. Analyze statements, branches, functions, and lines coverage to identify gaps.

5. Test Results

Test Execution:

Metric	Value
Test Suites Run	3
Total Tests	7
Passed	7
Failed	0
Snapshots	0
Execution Time	4.6 s

Observations:

- All critical backend APIs passed the tests successfully.
- Coverage report shows full coverage for models (100%) and lower coverage for controllers (19–45%), indicating additional tests needed for complete validation.
- MongoDB connection verified during tests, confirming live database interaction.

6. Figures

```
PS G:\Projects\ResQNow-Crowdsourced_Location_Aware_Emergency_Response_System\backend> npm test

> backend@1.0.0 test
> jest

 PASS  test/incident.test.js
  ● Console

    console.log
      [dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙️ override existing env vars with { override: true }

      at _log (node_modules/dotenv/lib/main.js:139:11)

    console.log
      MongoDB Connected: ac-fmhnzik-shard-00-01.2u6b9hq.mongodb.net

      at log (db.js:6:13)

 PASS  test/user.test.js
  ● Console

    console.log
      [dotenv@17.3.1] injecting env (3) from .env -- tip: 🔐 encrypt with Dotenvx: https://dotenvx.com

      at _log (node_modules/dotenv/lib/main.js:139:11)

    console.log
      MongoDB Connected: ac-fmhnzik-shard-00-00.2u6b9hq.mongodb.net

      at log (db.js:6:13)
```

```
 PASS  test/auth.test.js
  ● Console

    console.log
      [dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙️ write to custom object with { processEnv: myObject }

      at _log (node_modules/dotenv/lib/main.js:139:11)

    console.log
      MongoDB Connected: ac-fmhnzik-shard-00-01.2u6b9hq.mongodb.net

      at log (db.js:6:13)

    console.log
      REGISTER RESPONSE: { message: 'User registered successfully' }

      at Object.log (test/auth.test.js:22:15)


Test Suites: 3 passed, 3 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        4.592 s
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        4.592 s
Ran all test suites
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	61.3	41.66	26.31	61.3	
backend	72.72	50	25	72.72	
db.js	75	100	100	75	8-9
server.js	72	50	0	72	27,32-36,43-44
backend/controllers	44.73	50	22.22	44.73	
authController.js	89.28	87.5	100	89.28	33,56,73
incidentController.js	19.04	0	0	19.04	7-21,31-36,46-64
reportController.js	18.51	0	0	18.51	5-25,35-41,49-59,63-73
backend/middleware	45	16.66	66.66	45	
authMiddleware.js	50	50	100	50	11-21
roleMiddleware.js	37.5	0	50	37.5	5-14
backend/models	100	100	100	100	
Incident.js	100	100	100	100	
Report.js	100	100	100	100	
User.js	100	100	100	100	
backend/routes	90	100	0	90	
authRoutes.js	78.57	100	0	78.57	14,24,29
incidentRoutes.js	100	100	100	100	
reportRoutes.js	100	100	100	100	

7. Conclusion

- Backend functionalities are working as expected.
- Current test coverage is adequate for models but needs improvement for controllers and middleware.
- Future work: add more unit tests for incidentController, reportController, and middleware to reach higher coverage.